Java Strings — Beginner Teaching Guide

Clean expanded edition — useful content from the supplied reference PDF has been integrated.

Covered: String, String literals, String Constant Pool, literal vs new, constructors, core String methods, immutability, StringBuffer, StringBuilder, comparison, practice programs and viva questions.

1. String Basics

Definition: A String is a sequence of characters used to store text. In Java, String is a built-in class and a reference type.

The reference material covers three built-in classes for String data: String, StringBuffer and StringBuilder.

String Literal

Any data written inside double quotes is called a String literal. Examples: "hello", "Java", and "123".

```
String name = "Ravi";
String city = "Hyderabad";
System.out.println(name);
System.out.println(city);
```

Output:
Ravi
Hyderabad

Two Ways to Create a String

```
String a = "Java";
String b = new String("Java");
```

The literal form is the normal/simple form. The new form creates a separate String object in heap memory.

2. String Constructors

The reference PDF lists four String constructors: String(), String(String), String(StringBuffer), and String(StringBuilder).

```
String a = new String();
String b = new String("Java");
StringBuffer buffer = new StringBuffer("Hello");
String c = new String(buffer);
System.out.println(b);
System.out.println(c);
```

Output:
Java
Hello

3. String Constant Pool

When a String literal is used directly, Java can store it in the String Constant Pool. If the same literal already exists, the existing object can be reused.

```
String brand = "vivo";
String brand1 = "vivo";
System.out.println(brand == brand1);
```

Output:
true

Important: This example demonstrates reference reuse. For normal String content comparison, use equals().

Literal vs new

```
String a = "Java";
String b = "Java";
String c = new String("Java");

System.out.println(a == b);
System.out.println(a == c);
System.out.println(a.equals(c));
```

Output:
true
false
true

== checks references. equals() checks String content.

4. Core String Methods

length()

Definition: Counts characters.
Syntax: str.length()

```
String s = "Java";
System.out.println(s.length());
```

Output:
4

charAt()

Definition: Returns the character at an index.
Syntax: str.charAt(index)

```
String s = "Java";
System.out.println(s.charAt(2));
```

Output:
v

equals()

Definition: Compares String contents.
Syntax: s1.equals(s2)

```
String a = "Java";
String b = "Java";
System.out.println(a.equals(b));
```

Output:
true

substring()

Definition: Returns a part of a String.
Syntax: str.substring(start, end)

```
String s = "Programming";
System.out.println(s.substring(0, 7));
```

Output:
Program

toUpperCase()

Definition: Returns uppercase text.
Syntax: str.toUpperCase()

```
String s = "Java";
System.out.println(s.toUpperCase());
```

Output:
JAVA

toLowerCase()

Definition: Returns lowercase text.
Syntax: str.toLowerCase()

```
String s = "JAVA";
System.out.println(s.toLowerCase());
```

Output:
java

trim()

Definition: Removes leading/trailing whitespace.
Syntax: str.trim()

```
String s = " Java ";
System.out.println(s.trim());
```

Output:
Java

contains()

Definition: Checks whether text occurs inside the String.
Syntax: str.contains("text")

```
String s = "I love Java";
System.out.println(s.contains("Java"));
```

Output:
true

concat()

Definition: Joins another String at the end.
Syntax: str.concat("text")

```
String a = "Hello";
System.out.println(a.concat(" Java"));
```

Output:
Hello Java

isEmpty()

Definition: Checks whether length is zero.
Syntax: str.isEmpty()

```
String s = "";
System.out.println(s.isEmpty());
```

Output:
true

5. String Immutability

String is immutable. An existing String object cannot be changed. A String operation that produces modified text returns a new String.

```
String s = "Hello";
s.concat(" Java");
System.out.println(s);
```

Output:
Hello

The result of concat() was not assigned back to s.

```
String s = "Hello";
s = s.concat(" Java");
System.out.println(s);
```

Output:
Hello Java

6. StringBuffer

StringBuffer is mutable. Its methods can modify the same object. The reference material also notes that StringBuffer objects are directly created in heap memory.

append()

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" Java");
System.out.println(sb);
```

Output:
Hello Java

insert()

```
StringBuffer sb = new StringBuffer("Java");
sb.insert(4, " Programming");
System.out.println(sb);
```

Output:
Java Programming

replace()

```
StringBuffer sb = new StringBuffer("Java123");
sb.replace(4, 7, "XYZ");
System.out.println(sb);
```

Output:
JavaXYZ

delete()

```
StringBuffer sb = new StringBuffer("Java123");
sb.delete(4, 7);
System.out.println(sb);
```

Output:
Java

deleteCharAt()

```
StringBuffer sb = new StringBuffer("Java");
sb.deleteCharAt(1);
System.out.println(sb);
```

Output:
Jva

reverse()

```
StringBuffer sb = new StringBuffer("Java");
sb.reverse();
System.out.println(sb);
```

Output:
avaJ

length() and charAt()

```
StringBuffer sb = new StringBuffer("Java");
System.out.println(sb.length());
System.out.println(sb.charAt(1));
```

Output:
4
a

Synchronization: The reference notes say StringBuffer methods are synchronized. This can introduce coordination overhead, so StringBuilder is used when synchronization is not required.

7. StringBuilder

StringBuilder is mutable. The reference material states that its common methods are also present in StringBuffer, but StringBuilder methods are not synchronized.

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" Programming");
sb.reverse();
System.out.println(sb);
```

Output:
gnimmargorP avaJ

8. String vs StringBuffer vs StringBuilder

Feature	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable
Modification	New String result	Same object	Same object
Synchronization	Not applicable	Synchronized	Not synchronized
Typical use	Normal text	Mutable text + synchronization	Mutable text without synchronization

9. Beginner Practice Programs

Program 1 — Count Characters

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String name = sc.nextLine();
        System.out.println("Length = " + name.length());
    }
}
```

Output:
Input: Ramesh
Length = 6

Program 2 — Check for a Space

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String name = sc.nextLine();
        System.out.println(name.contains(" "));
    }
}
```

Output:
Input: Ravi Kumar
true

Program 3 — Build a Sentence

```
class Main {
    public static void main(String[] args) {
```

```
        StringBuffer sb = new StringBuffer("Java");
        sb.append(" is");
        sb.append(" easy");
        sb.append(" to learn");
        System.out.println(sb);
    }
}
```

Output:
Java is easy to learn

Program 4 — Remove One Character

```
class Main {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Jvaa");
        sb.deleteCharAt(2);
        System.out.println(sb);
    }
}
```

Output:
Jva

Program 5 — StringBuilder Changes

```
class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder();
        sb.append("A");
        sb.append("B");
        sb.append("C");
        System.out.println(sb);
    }
}
```

Output:
ABC

10. Viva Questions

Q: What is a String literal?
A: Text written inside double quotes, such as "Java".

Q: What is the String Constant Pool?
A: A special area associated with String literals where identical literal objects can be reused.

Q: What is the difference between literal creation and new String()?
A: A literal can reuse an existing pooled String; new creates a separate String object.

Q: Is String mutable?
A: No. String is immutable.

Q: Why do we use StringBuffer?
A: For repeated text modification when synchronized/thread-safe access is desired.

Q: What is deleteCharAt()?
A: It removes one character at a specified index from a mutable StringBuffer or StringBuilder.

Q: Why can StringBuffer have performance overhead?
A: Its methods are synchronized, which can introduce coordination overhead.

Q: Why can StringBuilder be faster?
A: Its methods are not synchronized, so it avoids that overhead when synchronization is unnecessary.

11. Teaching Flow

• String definition and literals

- Literal and new creation
- Constructors
- String Constant Pool
- Core String methods
- String immutability
- StringBuffer and common methods
- StringBuilder and common methods
- StringBuffer vs StringBuilder
- Practice programs
- Viva questions